Softmax Regression 📊

Softmax regression is the algorithm used for **multiclass classification** problems (where y can be one of N possible classes, with $N > 2$). It is a direct generalization of logistic regression.

From Logistic to Softmax Regression

To understand softmax, it helps to first re-frame logistic regression:

- **Logistic Regression (Binary: $N = 2$)**
  - Calculates one linear output: $z = \vec{w} \cdot \vec{x} + b$.
  - Calculates one probability: $a_1 = \text{sigmoid}(z) = P(y = 1)$.
  - The second probability is implicitly defined: $a_2 = 1 - a_1 = P(y = 0)$.
  - The probabilities a_1 and a_2 must sum to 1.
- **Softmax Regression (Multiclass: $N \geq 2$)**
  - The model must compute a separate probability for *each* of the N classes: $a_1, a_2, \ldots, a_N$.
  - These N probabilities must sum to 1.

The Softmax Regression Model

The model's computation is a two-step process:

Step 1: Compute Linear Outputs (z_j)

- Instead of one set of parameters, the model must learn **one set of parameters ($\vec{w}_j, b_j$) for each class j**.
- It computes a separate linear output (or "score") for each class:
  - $z_1 = \vec{w}_1 \cdot \vec{x} + b_1$
  - $z_2 = \vec{w}_2 \cdot \vec{x} + b_2$
  - ...
  - $z_N = \vec{w}_N \cdot \vec{x} + b_N$

Step 2: Compute Activations (a_j) using the Softmax Function

- We cannot use the sigmoid function, as the resulting probabilities would not sum to 1.
- We use the **softmax function**, which takes the vector of z-scores and converts it into a vector of probabilities.
- The formula for the activation (probability) of a single class j is:

$a_j = \frac{e^{z_j}}{\sum_{k=1}^{N} e^{z_k}}$

- **What this does:**
    1. By using e^{z_j}, it ensures all outputs are positive.
    2. By dividing by the sum of all e^{z_k}, it ensures the final probabilities $a_1, a_2, \ldots, a_N$ **all sum to 1**.
- **Interpretation:** a_j is the model's estimated probability that the true label y is equal to class j, given the input $\vec{x}$.

Loss Function for Softmax Regression 🧮

This model uses a loss function called **cross-entropy loss**.

- **Logistic Loss (Recap):** $L = -y \log(a_1) - (1-y) \log(a_2)$
- **Softmax (Cross-Entropy) Loss:** This generalizes the logistic loss. The loss for a single training example is calculated based *only* on the probability the model assigned to the **single correct class**.
    - If the true label $y = 1$, then Loss = $-\log(a_1)$
    - If the true label $y = 2$, then Loss = $-\log(a_2)$
    - ...
    - If the true label $y = j$, then Loss = $-\log(a_j)$
- **Intuition:** To make the loss small (close to 0), the model must learn to make the probability a_j for the *correct* class j as close to 1 as possible.
- **Cost Function:** The overall cost J is the average of this loss function L over all m training examples.

Note: Softmax regression with $N = 2$ is mathematically equivalent to logistic regression.